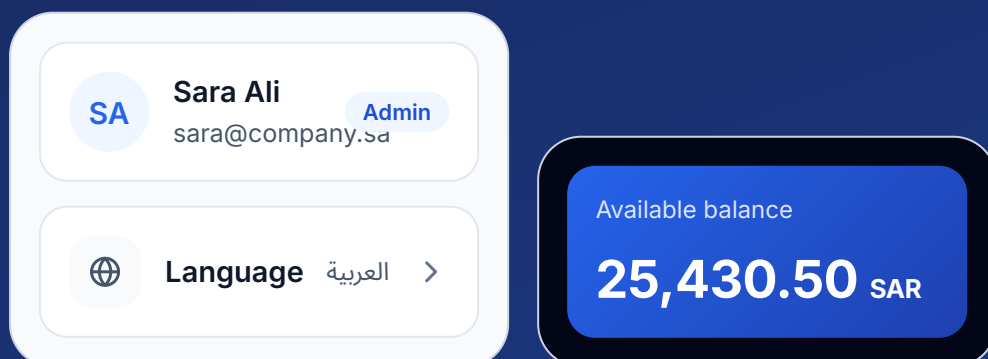


Preset components and screen builder

Ready-made cards, layout helpers and state helpers so screens contain data and logic instead of stacks of View, Text and Button. Each page shows the code you would write and the shape it renders.

Status: design proposal. Nothing here is built yet; names and props may change after review. Tick the boxes on the last page and send it back.



What is inside

Every preset is built from the existing Card, Line, List, Form and templates, so it follows the theme (light, dark, black, dim, sepia), Arabic RTL, accessibility and translations automatically. Texts accept a plain string or { localeKey }; every card takes loading to show a matching skeleton.

Preset cards

OneLineCard
TwoLineCard
ThreeLineCard
ProductCard
DetailsCard
AmountCard / StatCard
ProfileCard
SettingsGroup
StatusCard / ResultCard
ActionCard

Status feedback

StatusScreen
StatusModal
StatusToast · StatusBanner
Status animations

Layout helpers

Section

Row · Column · Center · Spacer · Grid

Less state code

useConfirm · useActionSheet · usePrompt

DataState

ListScreen

Wizard

Data display

Money · DateText · PhoneText · MaskedText

Screen builder

ScreenContent (blocks array)

Icons

How to import · categorized catalogue (153 icons)

Themes

Arabic RTL and dark preview

Your decision

Pick what to build first

Preset cards

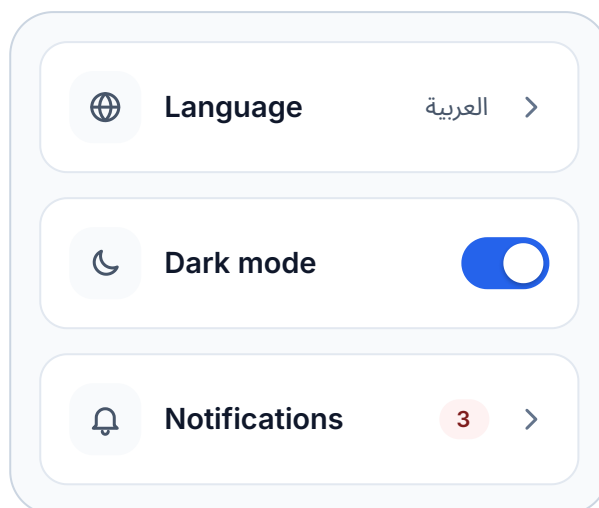
One component per common card shape. Pass data, get the card.

OneLineCard Proposed

One tappable row: icon or avatar, title and an optional trailing value, badge, switch or chevron.

Props: title · icon | avatar · value · badge · toggle · chevron · onPress · loading

```
<OneLineCard icon="globe" title="Language"
value="العربية" chevron onPress={openLanguage} />
<OneLineCard icon="moon" title="Dark mode" toggle=
{{ value: dark, onChange: setDark }} />
<OneLineCard icon="bell" title="Notifications"
badge={{ label: '3', tone: 'error' }} chevron />
```

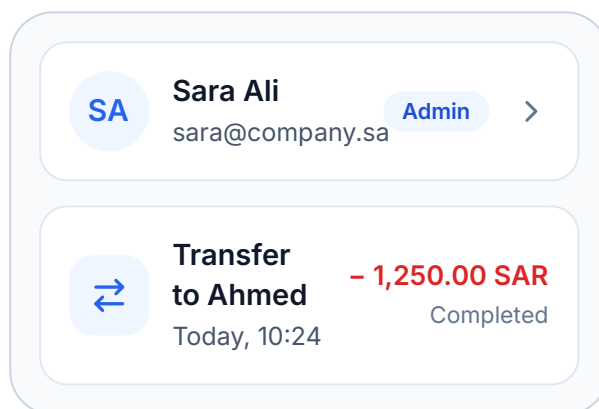


TwoLineCard Proposed

Title plus a second line (status, date, email...). The most common list row.

Props: title · subtitle · icon | avatar · value · subValue · badge · chevron · onPress · loading

```
<TwoLineCard
avatar={{ name: 'Sara Ali', image: sara.photo }}
title="Sara Ali"
subtitle="sara@company.sa"
badge={{ label: 'Admin', tone: 'primary' }}
chevron
onPress={() => openUser(sara.id)}
/>
<TwoLineCard icon="transfer" title="Transfer to
Ahmed" subtitle="Today, 10:24"
value="- 1,250.00 SAR" valueTone="error" />
```

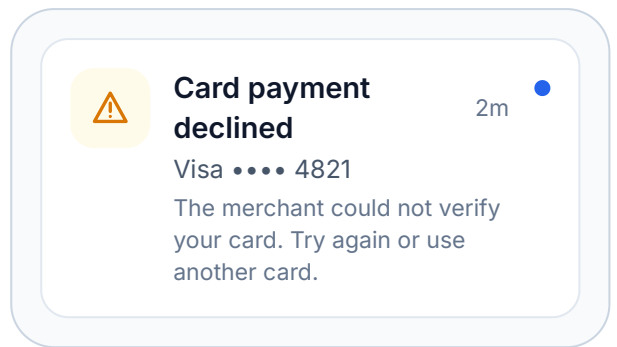


ThreeLineCard Proposed

Title, subtitle and a short description, for notifications, tickets or messages.

Props: title · subtitle · description · icon | avatar · meta (time) · badge · unread · onPress

```
<ThreeLineCard
  icon={{ name: 'warning', tone: 'warning' }}
  title="Card payment declined"
  subtitle="Visa •••• 4821"
  description="The merchant could not verify your
card. Try again or use another card."
  meta="2m"
  unread
/>
```



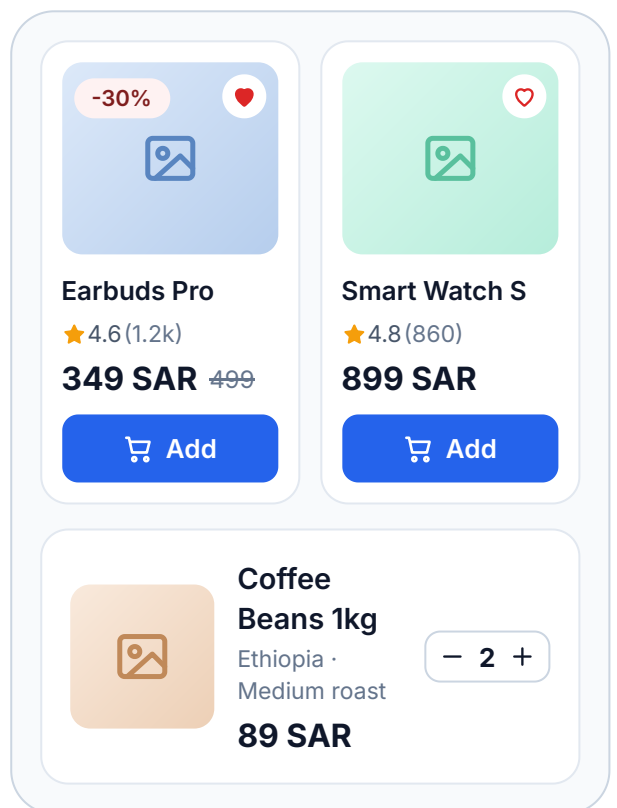
ProductCard Proposed

Image, title, price with optional old price, rating, badge and a call to action. Vertical (grid) and horizontal layouts.

Props: image · title · subtitle · price · oldPrice · currency · rating · reviews · badge · favorite · cta · layout: 'vertical' | 'horizontal'

```
<ProductCard
  image={product.image}
  title="Wireless Earbuds Pro"
  price={349} oldPrice={499} currency="SAR"
  rating={4.6} reviews={1280}
  badge={{ label: '-30%', tone: 'error' }}
  favorite={{ value: liked, onChange: setLiked }}
  cta={{ title: 'Add to cart', icon: 'cart',
onPress: add }}
/>

<List Component={ProductCard} data={products}
columns={2}
  formatItem={p => ({ image: p.image, title:
p.name, price: p.price, currency: 'SAR' })} />
```



DetailsCard Proposed

Label / value rows (receipt, account details, order summary) with copy, badges and a total.

Props: title · rows: [{ label, value, badge, copyable, tone, emphasis }] · total · footer · action

```
<DetailsCard
  title="Transfer details"
  rows=[
    { label: 'Beneficiary', value: 'Ahmed Saleh' },
    { label: 'IBAN', value: 'SA03 8000 0000 6080
1016 7519', copyable: true },
    { label: 'Reference', value: 'TRX-482193',
copyable: true },
    { label: 'Status', badge: { label: 'Completed',
tone: 'success' } },
    { label: 'Fees', value: <Money amount={0} /> },
  ]
  total={{ label: 'Total', value: <Money amount=
{1250} currency="SAR" /> }}
/>
```

Transfer details

Beneficiary	Ahmed Saleh
IBAN	SA03 8000 ... 7519 
Reference	TRX-482193 
Status	Completed
Fees	0.00 SAR
Total	1,250.00 SAR

AmountCard / StatCard Proposed

A big number with label, currency, trend and quick actions: balances, KPIs, dashboards.

Props: label · amount · currency · trend: { value, direction } · hidden (eye toggle) · actions · variant: 'primary' | 'surface'

```
<AmountCard
  variant="primary"
  label="Available balance"
  amount={25430.5} currency="SAR"
  trend={{ value: '+4.2%', direction: 'up' }}
  hideable
  actions=[
    { title: 'Transfer', icon: 'transfer', onPress:
goTransfer },
    { title: 'Top up', icon: 'plus', onPress:
goTopUp },
  ]
/>
<Row gap="md">
  <StatCard label="Orders" value={128} trend={{
value: '+12', direction: 'up' }} />
  <StatCard label="Refunds" value={3} trend={{
value: '-2', direction: 'down' }} />
</Row>
```

Available balance

▲ +4.2%

25,430.50 SAR

↔ Transfer

+ Top up

Orders

128

▲ +12 this week

Refunds

3

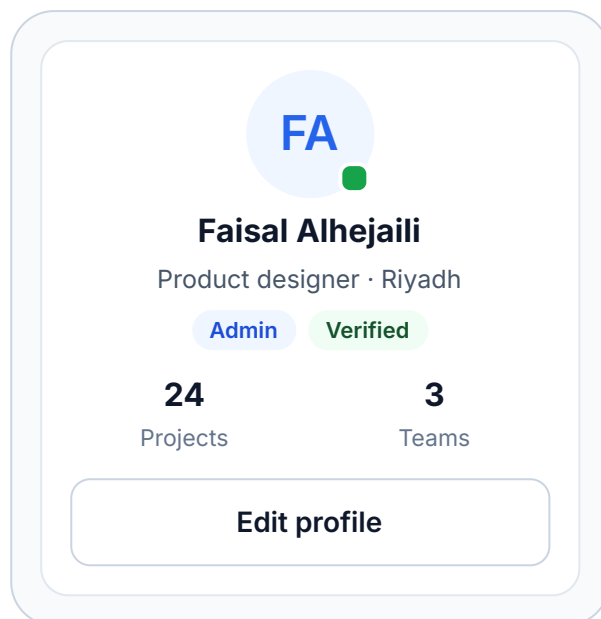
▼ -2 this week

ProfileCard Proposed

Avatar, name, role, badges and actions: account headers, team members, contacts.

Props: avatar · name · subtitle · badges · stats: [{ label, value }] · actions · layout: 'horizontal' | 'centered'

```
<ProfileCard
  layout="centered"
  avatar={{ name: 'Faisal Alhejaili', image:
me.photo, status: 'online' }}
  name="Faisal Alhejaili"
  subtitle="Product designer · Riyadh"
  badges=[[{ label: 'Admin', tone: 'primary' }, {
label: 'Verified', tone: 'success' }]]
  stats=[[{ label: 'Projects', value: 24 }, {
label: 'Teams', value: 3 }]]
  actions=[[{ title: 'Edit profile', variant:
'outline', onPress: edit }]]
/>
```

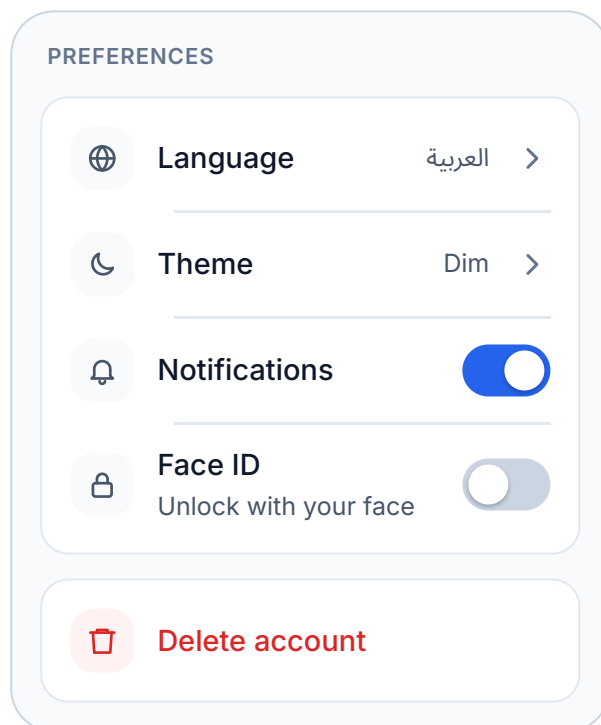


SettingsGroup Proposed

A titled group of setting rows with switches, values and chevrons. A whole settings screen is a few groups.

Props: title · footer · items: [{ icon, title, subtitle, value, toggle, chevron, badge, destructive, onPress }]

```
<SettingsGroup
  title="Preferences"
  items=[
    { icon: 'globe', title: 'Language', value:
'العربية', chevron: true, onPress: pickLanguage },
    { icon: 'moon', title: 'Theme', value: 'Dim',
chevron: true, onPress: pickTheme },
    { icon: 'bell', title: 'Notifications', toggle:
{ value: push, onChange: setPush } },
    { icon: 'lock', title: 'Face ID', subtitle:
'Unlock with your face', toggle: { value: bio,
onChange: setBio } },
  ]
/>
<SettingsGroup items=[[{ icon: 'trash', title:
'Delete account', destructive: true, onPress:
confirmDelete }]] />
```

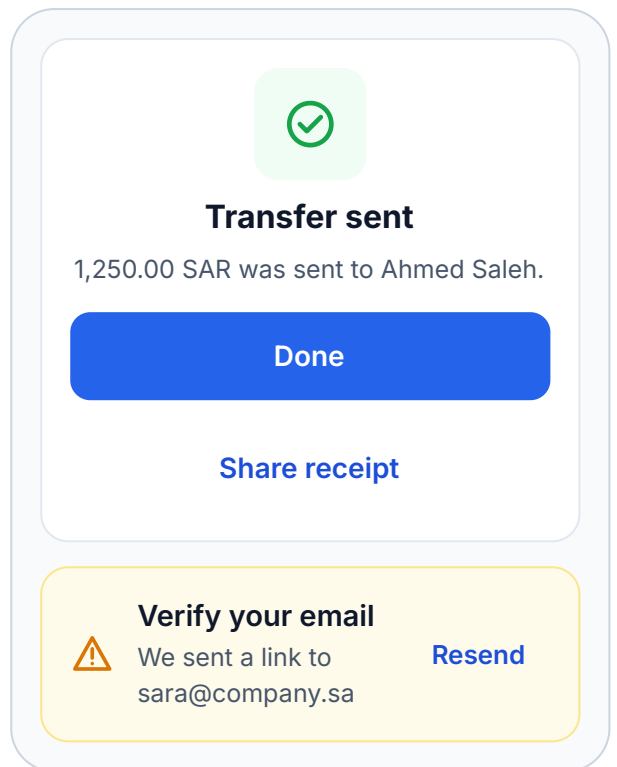


StatusCard / ResultCard Proposed

Success, warning, error or info message with an icon and actions, inline or full screen.

Props: status: 'success' | 'warning' | 'error' | 'info' · title · message · details (DetailsCard rows) · primaryAction · secondaryAction

```
<StatusCard
  status="success"
  title="Transfer sent"
  message="1,250.00 SAR was sent to Ahmed Saleh."
  primaryAction={{ title: 'Done', onPress: goHome
}}
  secondaryAction={{ title: 'Share receipt',
onPress: share }}
/>
<StatusCard status="warning" title="Verify your
email" message="We sent a link to sara@company.sa"
  primaryAction={{ title: 'Resend', variant:
'ghost', onPress: resend }} compact />
```

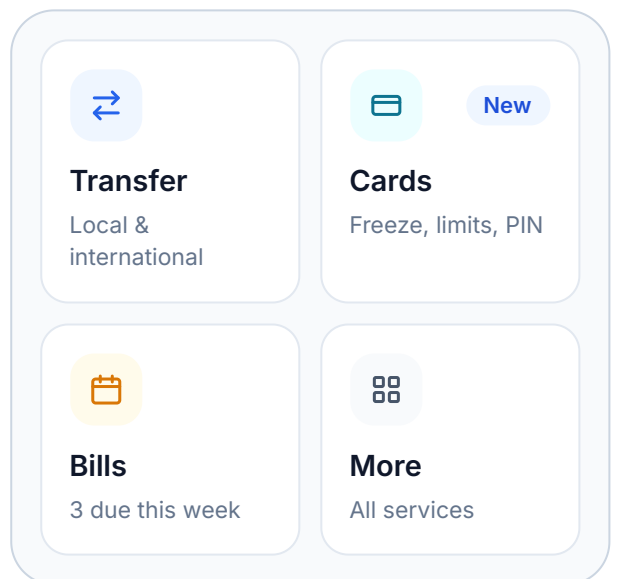


ActionCard Proposed

Large tappable tiles for menus, dashboards and onboarding choices. Use in a grid.

Props: icon · title · description · badge · tone · selected · onPress

```
<Grid columns={2} gap="md">
  <ActionCard icon="transfer" title="Transfer"
description="Local & international" onPress=
{goTransfer} />
  <ActionCard icon="card" title="Cards"
description="Freeze, limits, PIN" badge={{ label:
'New' }} onPress={goCards} />
  <ActionCard icon="calendar" title="Bills"
description="3 due this week" tone="warning"
onPress={goBills} />
  <ActionCard icon="grid" title="More"
description="All services" onPress={goMore} />
</Grid>
```



Status feedback

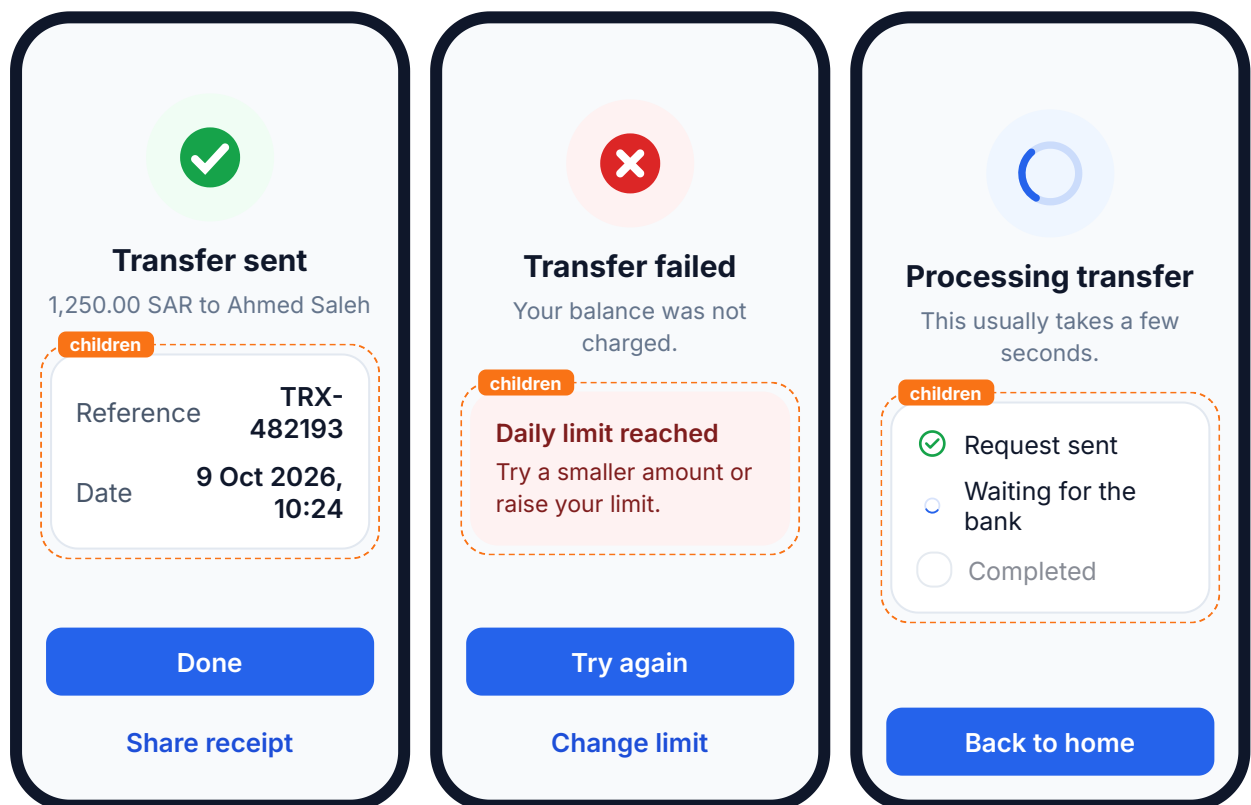
One family for every result: full screen, modal, toast and banner, all with success, error and pending, the same props and the same animations.

StatusScreen Proposed

Full-screen result for success, error and pending. Title, subtitle, your own content between the text and the buttons, and up to two buttons. The icon animates; pending loops until the status changes.

Props: status: 'success' | 'error' | 'pending' · title · subtitle · children (between subtitle and buttons) · primaryAction · secondaryAction · animated (default true) · icon / animation (your own icon or Lottie) · onAnimationEnd

```
<StatusScreen
  status={state} // 'pending' → 'success' | 'error'
  title={state === 'success' ? 'Transfer sent' : state === 'error' ? 'Transfer failed' :
'Processing transfer'}
  subtitle="1,250.00 SAR to Ahmed Saleh"
  primaryAction={{ title: 'Done', onPress: goHome }}
  secondaryAction={{ title: 'Share receipt', onPress: share }}
>
  <DetailsCard rows={[{ label: 'Reference', value: 'TRX-482193' }, { label: 'Date', value:
<DateText value={now} /> }]} />
</StatusScreen>
```

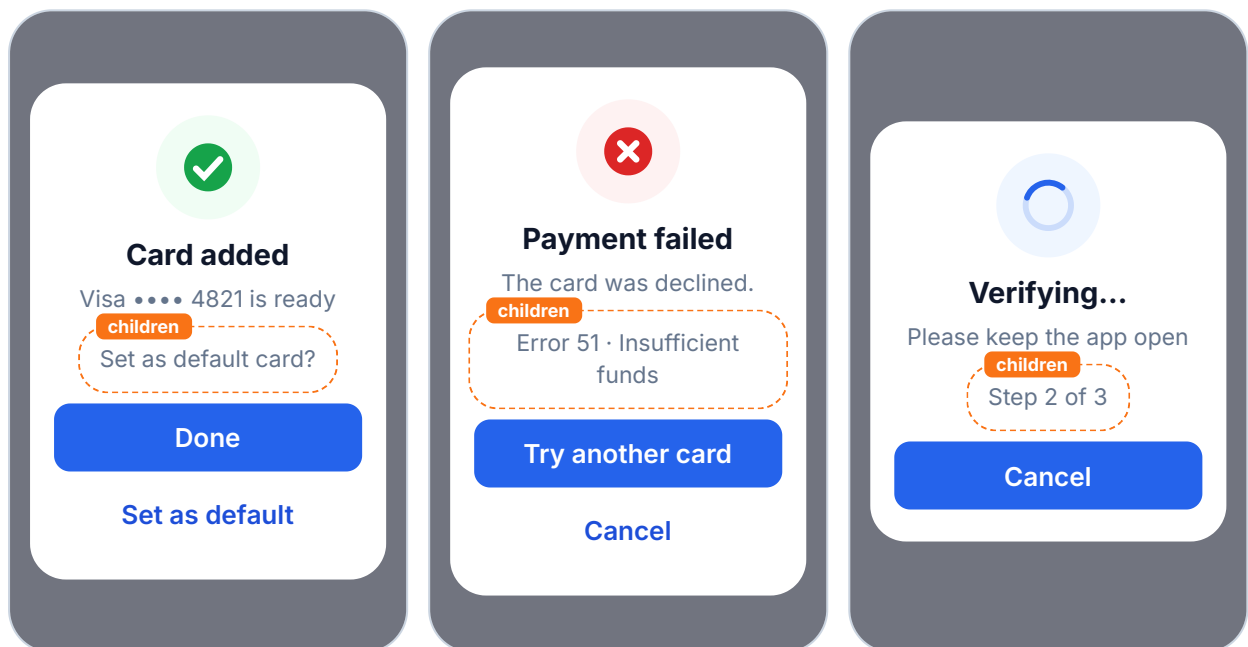


StatusModal Proposed

The same content in a dialog over the current screen. Title, subtitle, children and two buttons. Pending cannot be dismissed until it finishes.

Props: visible · status · title · subtitle · children · primaryAction · secondaryAction · dismissible (false while pending) · onClose · animated

```
<StatusModal
  visible={open}
  status="error"
  title="Payment failed"
  subtitle="The card was declined by the bank."
  primaryAction={{ title: 'Try another card', onPress: pickCard }}
  secondaryAction={{ title: 'Cancel', onPress: close }}
  onClose={close}
>
  <Text value="Error code 51 · Insufficient funds" variant="caption" tone="secondary"
  align="center" />
</StatusModal>
```



StatusToast · StatusBanner Proposed

StatusToast floats over the screen and can change from pending to success or error in place. StatusBanner is an inline bar at the top of a screen or section. (Called "Banner" so it does not clash with React Native's StatusBar.)

Props: toast: status · title · subtitle · action · duration (pending stays until updated) · placement | show() → { update, dismiss } · run(promise, texts) | banner: status · title · action · dismissible · animated

```
const status = useStatusToast();

// Show pending, then update the same toast
const t = status.show({ status: 'pending', title:
'Uploading 3 files...' });
try {
  await upload(files);
  t.update({ status: 'success', title: 'Files
uploaded' });
} catch (e) {
  t.update({ status: 'error', title: 'Upload
failed', action: { title: 'Retry', onPress: retry }
});
}

// Or in one line
await status.run(saveProfile(values), {
  pending: 'Saving...', success: 'Profile saved',
  error: e => e.message,
});

<StatusBanner status="error" title="No internet
connection" action={{ title: 'Retry', onPress:
reconnect }} />
```

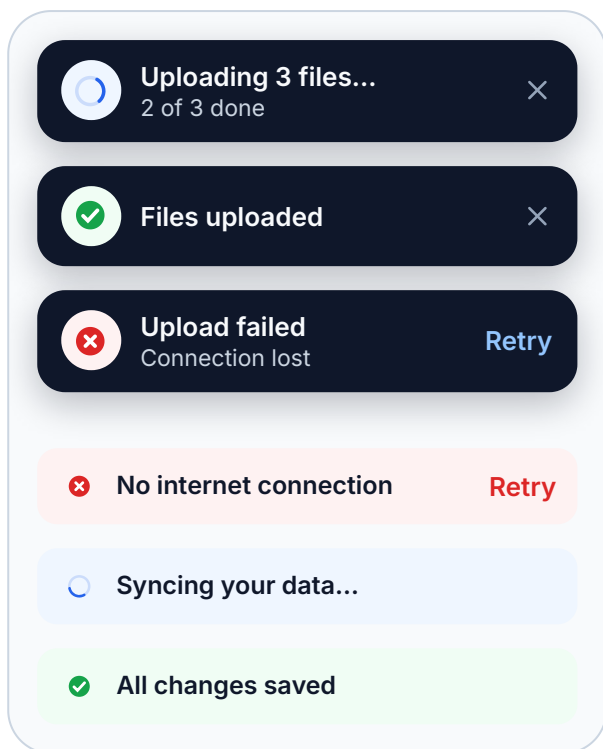
Status animations Proposed

Built-in motion for every status component. It respects the system Reduce Motion setting (shows the final frame), and any status can use your own Lottie or icon instead.

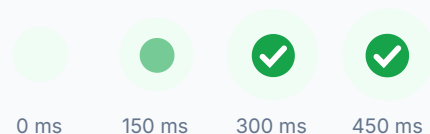
Props: animated · animation: 'default' | 'none' | ReactNode (e.g. <LottieView source={...} />) · loop (pending) · haptics: success / error feedback · onAnimationEnd

```
// success: circle scales in, then the check is
drawn (450 ms, light haptic)
// error: circle scales in, the X is drawn, the
icon shakes once (error haptic)
// pending: ring spins until `status` changes, then
morphs into success / error

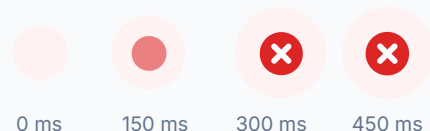
<StatusScreen status="success" title="Done"
animation=<LottieView source={confetti} autoPlay
loop={false} /> />
<StatusToast status="pending" title="Sending..."
animated={false} />
```



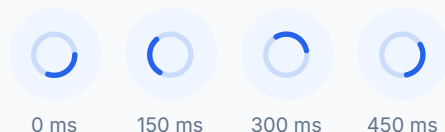
Success: scale in, draw check



Error: scale in, draw X, shake



Pending: spinning ring (loops)



Layout helpers

For the few places you still lay things out yourself.

Section Proposed

Title row with an optional action, then content. Shows its own empty state and skeleton.

Props: title · subtitle · action · loading · empty (EmptyState props) · children

```
<Section title="Recent transfers" action={{ title:
'See all', onPress: goHistory }}
  loading={isLoading} empty={{ title: 'No transfers
yet', icon: 'transfer' }}>
  <List Component={TwoLineCard} data={transfers}
  formatItem={toRow} />
</Section>
```

Recent transfers

[See all](#)

AS	Ahmed Saleh Today	-1,250
NO	Noura Omar Yesterday	+400

Saved cards

[Add](#)

No cards yet

Add a card to pay faster

Row · Column · Center · Spacer · Grid Proposed

Layout without style objects: spacing, padding and colours are theme token names. RTL is automatic.

Props: gap · p · px · py · bg ('surface' | 'subtle' | ...) · radius · align · justify · wrap · flex · columns (Grid)

```
<Column gap="lg" p="xl" bg="surface" radius="lg">
  <Row gap="sm" align="center">
    <Avatar name="Sara Ali" />
    <Column flex={1}>
      <Text value="Sara Ali" weight="semibold" />
      <Text value="Online" tone="success"
variant="caption" />
    </Column>
    <IconButton icon="phone"
accessibilityLabel="Call" />
  </Row>
  <Grid columns={{ compact: 2, medium: 4 }}
gap="md">...</Grid>
  <Spacer />
  <Center><Button title="Continue" /></Center>
</Column>
```



Less state code

Remove visible/loading/error state and modal JSX from screens.

useConfirm · useActionSheet · usePrompt Proposed

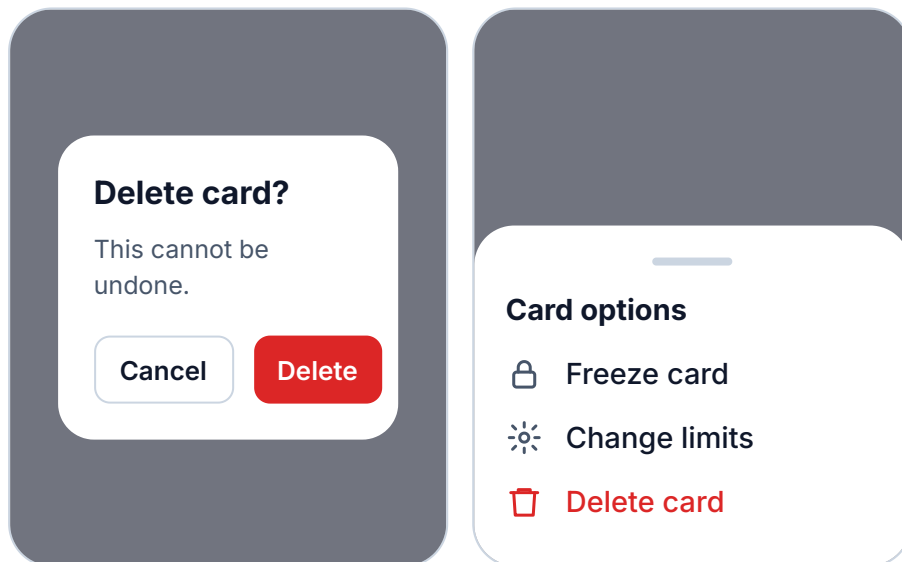
Dialogs and sheets you call like a function. No visible state, no onClose, no modal JSX in the screen.

Props: confirm({ title, message, confirmText, cancelText, danger }) → Promise<boolean> · actionSheet(options) → Promise<value | null> · prompt({ title, field }) → Promise<string | null>

```
const confirm = useConfirm();
const actionSheet = useActionSheet();

async function onDelete() {
  if (await confirm({ title: 'Delete card?', message: 'This cannot be undone.', danger: true })) {
    await deleteCard(id);
  }
}

async function onMore() {
  const choice = await actionSheet({
    title: 'Card options',
    options: [
      { value: 'freeze', title: 'Freeze card', icon: 'lock' },
      { value: 'limits', title: 'Change limits', icon: 'settings' },
      { value: 'delete', title: 'Delete card', icon: 'trash', destructive: true },
    ],
  },
  );
  if (choice === 'delete') onDelete();
}
```



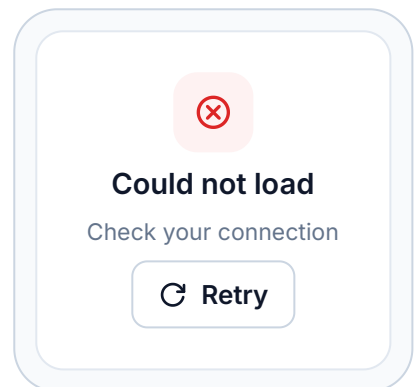
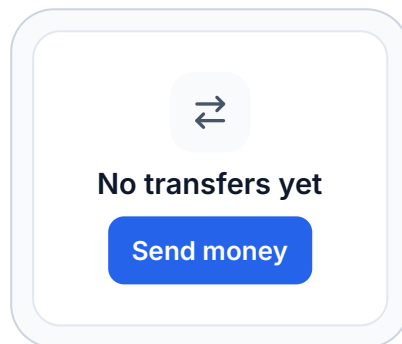
DataState Proposed

One wrapper for loading, empty, error and retry around fetched data. Works with React Query results.

Props: query (React Query result) | loading · error · data · skeleton (preset name or element) · empty · onRetry · children(data)

```
const transfers = useQuery({ queryKey: ['transfers'], queryFn: fetchTransfers });

<DataState
  query={transfers}
  skeleton="TwoLineCard" // matching skeleton, repeated
  empty={{ icon: 'transfer', title: 'No transfers yet', action: { title: 'Send money', onPress:
goSend } }}
>
  {data => <List Component={TwoLineCard} data={data} formatItem={toRow} />}
</DataState>
```



ListScreen Proposed

A complete list screen from config: search, filter chips, sorting, pull to refresh, pagination, empty and error states.

Props: title · data | query · renderAs (preset) · formatItem · search · filters · sort · onRefresh · onEndReached · groupBy · empty

```
<ListScreen
  title="Transactions"
  query={transactions}
  renderAs="TwoLineCard"
  formatItem={t => ({ icon: t.icon, title: t.name,
    subtitle: formatTime(t.date),
                        value: <Money amount=
{t.amount} signed /> )}}
  search={{ placeholder: 'Search transactions',
keys: ['name', 'reference'] }}
  filters=[[{ name: 'type', options: [{ label:
'All', value: 'all' },
    { label: 'Incoming', value: 'in' }, { label:
'Outgoing', value: 'out' }] }]]
  groupBy={t => formatDay(t.date)}
  onEndReached={transactions.fetchNextPage}
/>
```

< Transactions

🔍 Search transactions

All

Incoming

Outgoing

TODAY



Salary

08:00

+18,000.00



Coffee shop

09:41

-24.00

YESTERDAY



Grocery store

19:12

-312.50

Wizard Proposed

Multi-step flow where each step is a Form fields array with its own Yup schema. Progress, back and next are handled.

Props: steps: [{ title, description, fields, validationSchema }] · initialValues · onSubmit · showProgress · submitText

```
<Wizard
  initialValues={{ name: '', mobile: '', iban: '',
amount: null }}
  onSubmit={createBeneficiary}
  steps=[
    { title: 'Who', fields: [
      { type: 'InputField', name: 'name', label:
'Full name' },
      { type: 'PhoneWithCountryInput', name:
'mobile', label: 'Mobile' } ],
      validationSchema: Yup.object({ name:
Yup.string().required(), mobile:
Yup.string().mobileNumber() }) },
    { title: 'Bank', fields: [{ type: 'InputField',
name: 'iban', label: 'IBAN' }],
      validationSchema: Yup.object({ iban:
Yup.string().iban('SA').required() }) },
    { title: 'Review', fields: [<WizardSummary
key="s" />] },
  ]}
/>
```

< Add beneficiary

Step 2 of 3 · Bank

SA03 8000 0000 6080 1016 7519

✓ Al Rajhi Bank

Back

Next

Data display

Consistent formatting of money, dates, phones and masked numbers.

Money · DateText · PhoneText · MaskedText

Proposed

Formatted values that follow the locale, with optional Arabic-Indic digits. No formatting helpers in screens.

Props: Money: amount · currency · signed · decimals · compact · hidden | DateText: value · format ('date' | 'time' | 'relative' | 'hijri') | MaskedText: type ('iban' | 'card' | 'phone') · reveal

```
<Money amount={-1250} currency="SAR" signed />
// -1,250.00 SAR
<Money amount={2400000} compact />
// 2.4M
<DateText value={tx.date} format="relative" />
// 5 minutes ago
<DateText value={tx.date} format="hijri" />
// 12 Rabi II 1448
<PhoneText value="+966512345678" />
// +966 51 234 5678
<MaskedText type="card" value="4821..." />
// .... 4821
<MaskedText type="iban" value={iban} reveal={show} />
```

<Money signed>

-1,250.00 SAR

<Money compact>

2.4M

relative

5 minutes ago

hijri

12 Rabi II 1448

<PhoneText>

+966 51 234 5678

card

.... 4821

iban

SA03 7519

المبلغ

١٢٥٠,٠٠ ر.س-

التاريخ

منذ ه دقائق

هجري

١٢ ربيع الآخر ١٤٤٨

Screen builder: ScreenContent Proposed

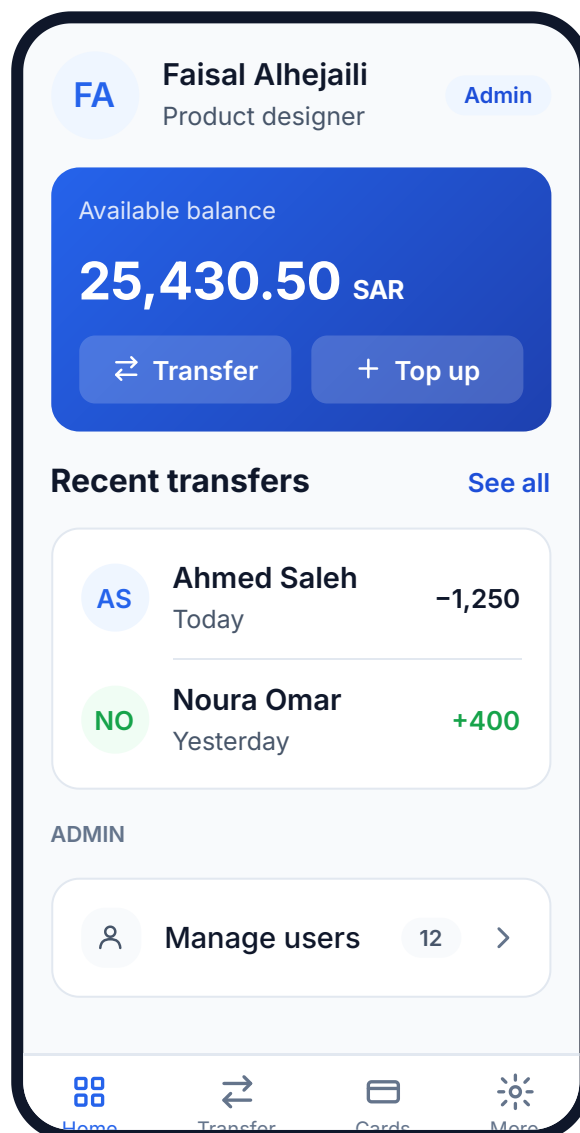
Describe a whole screen as a blocks array, in the same style as `<Form fields>`. Every preset above is a block type; add your own with `registerBlockType`. Later the same JSON can come from your backend (server-driven UI).

Today: layout code in every screen

```
<View style={styles.container}>
  <View style={styles.header}>
    <Image source={{ uri: user.photo }} style={styles.avatar} />
    <View style={{ flex: 1 }}>
      <Text style={styles.name}>{user.name}</Text>
      <Text style={styles.role}>{user.role}</Text>
    </View>
    {user.isAdmin && (
      <View style={styles.badge}><Text style={styles.badgeText}>Admin</Text></View>
    )}
  </View>
  <View style={styles.balanceCard}>
    <Text style={styles.label}>Available balance</Text>
    <Text style={styles.amount}>{format(balance)} SAR</Text>
    <View style={styles.row}>{/* 2 Pressables with icons */}</View>
  </View>
  {loading ? <ActivityIndicator /> : error ? <ErrorView ... /> : (
    transfers.map(t => <View key={t.id} style={styles.item}>...</View>)
  )}
</View>
// + 60 lines of StyleSheet.create({...})
```

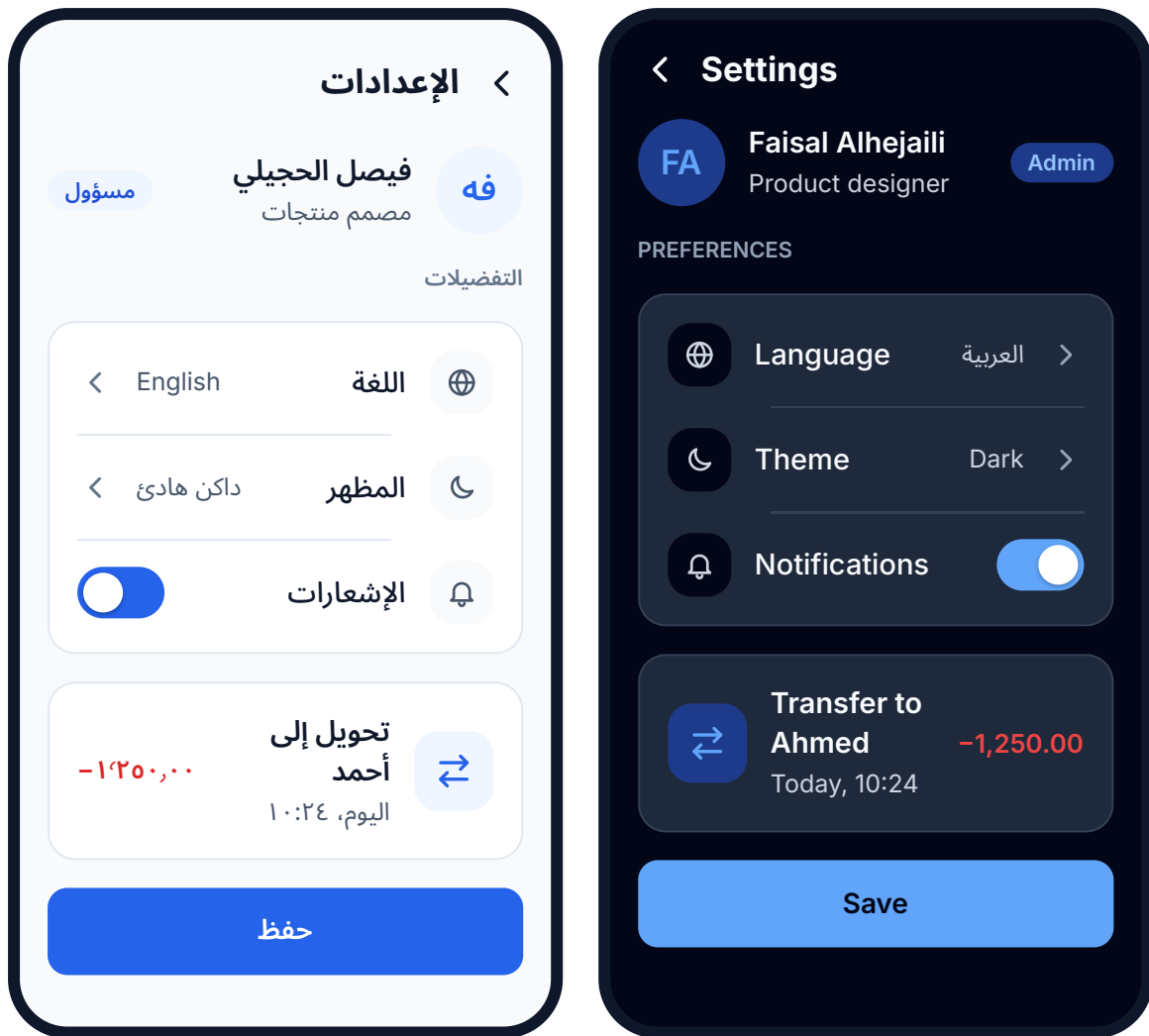
With ScreenContent

```
<ScreenContent
  blocks={[
    { type: 'ProfileCard', avatar: { name: user.name,
      image: user.photo },
      name: user.name, subtitle: user.role,
      badges: user.isAdmin && [{ label: 'Admin',
        tone: 'primary' }] },
    { type: 'AmountCard', variant: 'primary', label:
      'Available balance',
      amount: balance, currency: 'SAR',
      actions: [{ title: 'Transfer', icon:
        'transfer', onPress: goTransfer },
        { title: 'Top up', icon: 'plus',
          onPress: goTopUp }] },
    { type: 'Section', title: 'Recent transfers',
      action: { title: 'See all', onPress: goHistory
    } },
    query: transfers, renderAs: 'TwoLineCard',
    formatItem: toRow },
    isAdmin && { type: 'SettingsGroup', title:
      'Admin', items: adminItems },
  ]}
/>
// No StyleSheet. Falsy entries are skipped.
// Custom blocks: registerBlockType('PromoBanner',
PromoBanner)
```



Arabic RTL and dark theme

The same blocks, no extra code: the layout mirrors for Arabic (icons, chevrons, switches, dividers), numbers stay left-to-right, and colours come from the active theme mode.



Icons Proposed

153 icons in 12 categories, all in one style so they sit well together. Any app that uses Bunyan imports them from the package; no SVG files to copy. Icons marked with a blue dot already exist in 0.0.3.

```
// 1. By name (as today). IconName autocompletes every icon below.
import { Icon } from '@bunyan/design-system';
<Icon name="wallet" size="md" tone="primary" />

// 2. One component per icon: only what you import ends up in the app bundle.
import { WalletIcon, ReceiptIcon } from '@bunyan/design-system/icons';
<WalletIcon size={24} color={theme.color.primary.default} />

// 3. Categories, e.g. for an icon picker or a settings screen.
import { iconCategories } from '@bunyan/design-system/icons';
iconCategories.finance; // ['wallet', 'card', 'bank', 'transfer', ...]

// 4. Add your own (logo, product glyphs). Typed, works with <Icon name>.
registerIcons({ 'streamspy-logo': <Path d="M4 12h16..." /> });
declare module '@bunyan/design-system' {
  interface CustomIcons { 'streamspy-logo': true }
}
```

Grid	24 × 24, 2 px keyline padding
Stroke	2 px (theme borderWidth.medium), round caps and joins
Sizes	xs 12 · sm 16 · md 20 · lg 24 · xl 32 · xxl 40
Colour	tone (primary, secondary, success...) or any colour
RTL	directional icons (arrows, chevrons, send, log-in/out) mirror in Arabic automatically
Accessibility	decorative by default; pass accessibilityLabel to announce

Icon catalogue

Names are what you pass to <Icon name>. Blue dot = already in 0.0.3.

Navigation & arrows

iconCategories.navigation



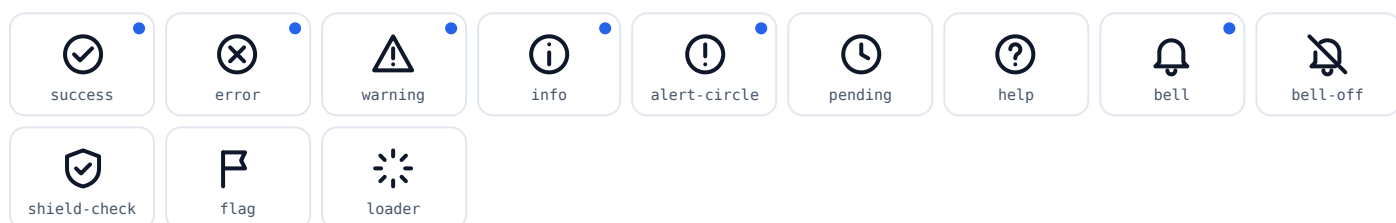
Actions

iconCategories.actions



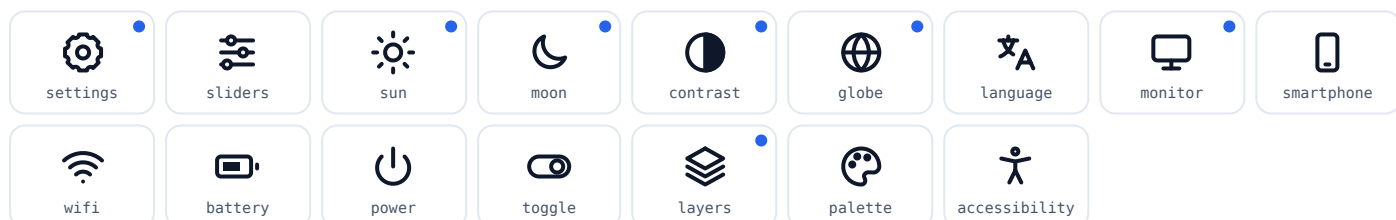
Status & alerts

iconCategories.status



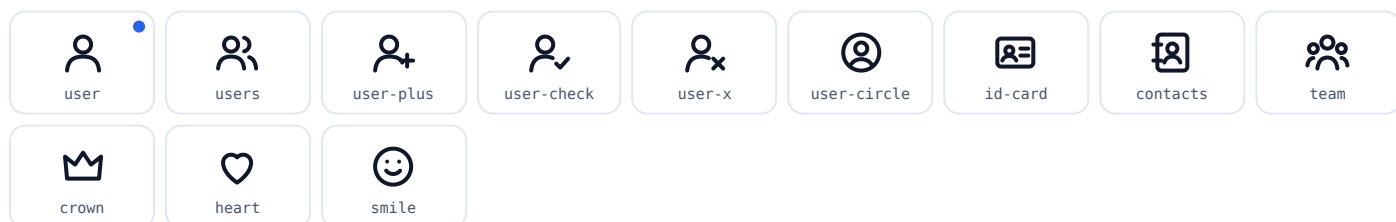
Settings & system

iconCategories.settings



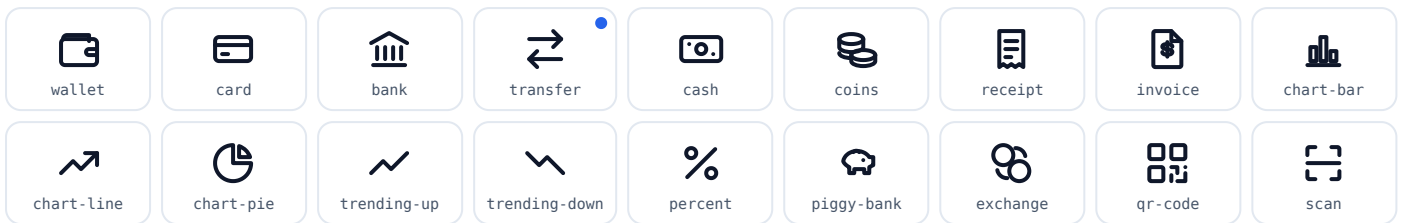
Users & people

iconCategories.users



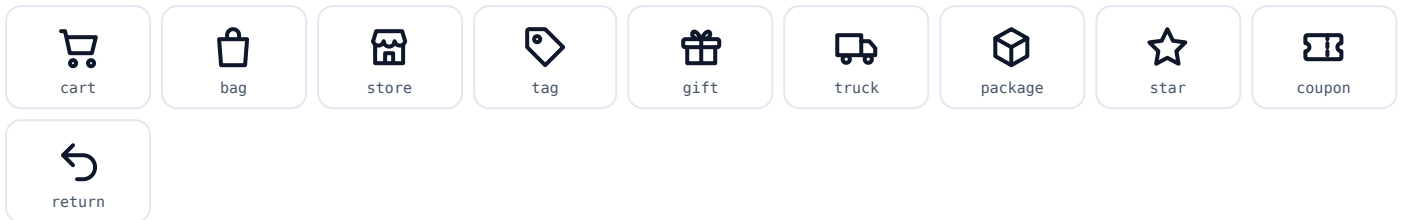
Finance & banking

iconCategories.finance



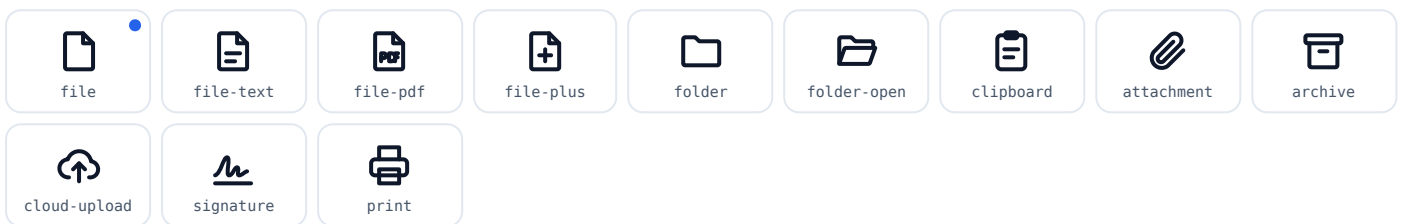
Commerce

iconCategories.commerce



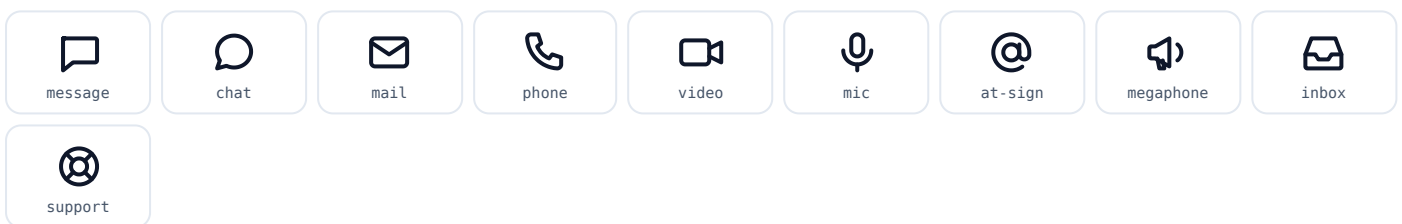
Files & documents

iconCategories.files



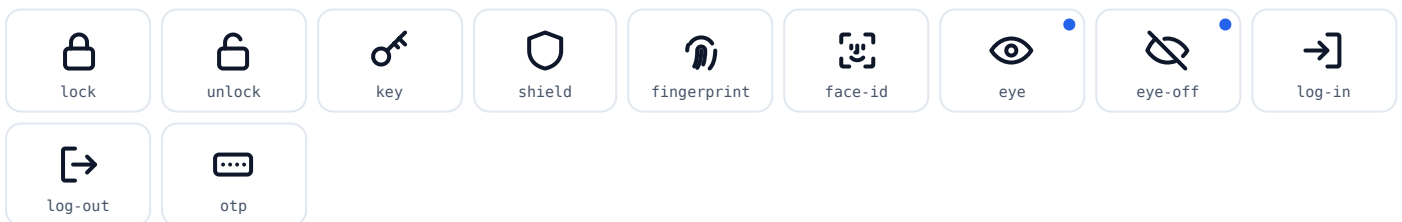
Communication

iconCategories.communication



Security & access

iconCategories.security



Time, place & travel

iconCategories.time


calendar


clock


history


timer


map-pin


map


navigation


plane


car


building

Media

iconCategories.media


image


camera


play


pause


volume


music


stream


cast

Your decision

Suggested order. Tick what you want, cross what you do not, add notes, and send this page back. Size: S ≈ 1 day, M ≈ 2–3 days, L ≈ a week.

	PHASE	COMPONENT	WHY	SIZE	NOTES
☐	1	OneLine / TwoLine / ThreeLine cards	Most rows in every app	S	
☐	1	SettingsGroup	Whole settings screens	S	
☐	1	Section · Row · Column · Grid	Removes style objects	S	
☐	1	useConfirm · useActionSheet	Removes modal state	S	
☐	2	ProductCard · DetailsCard · AmountCard · StatCard	Commerce and banking screens	M	
☐	2	ProfileCard · StatusCard · ActionCard	Headers, results, menus	M	
☐	2	DataState	Loading / empty / error everywhere	S	
☐	2	Money · DateText · PhoneText · MaskedText	Consistent formatting	S	
☐	1	StatusScreen · StatusModal · StatusToast · StatusBanner	One result family: success / error / pending, animated	M	
☐	1	Icon catalogue + @bunyan/design-system/icons	No more copied SVGs; categories for pickers	M	
☐	3	ScreenContent	Screens as data	M	
☐	3	ListScreen · Wizard	Full screens from config	L	
☐	4	Server-driven blocks (JSON from backend)	Change screens without a release	L	